

lab02 AI Prompts:

Depth Analyzer:

Prompt 1: Analyze the past data that contains the reservoir stage record with a logger every 15 minutes. 1. get the function of the data series 2. smoothen the curve from the derivatives from the fit

Claude: What the log actually shows

Your Data01.xlsx "Sensor Log" sheet is 288 readings at 15-min spacing — three days (0 to 71.75 h from the first timestamp), not the single 96-reading day the activity sheet describes. The shape is a slow pre-event creep (14.18 → ~14.5 m over 25 h), a sharp rise to a crest of ~21.1 m around $t \approx 36$ h, then a recession that overshoots down and settles to a new plateau near 19.5 m — a classic flood pulse, not a simple monotonic fill.

1. The function of the data series

A single logistic can't produce that overshoot-and-settle behavior, so I used the lab's "sum of two logistics" candidate — one rising limb for the flood rise, a second (negative-amplitude, slower) limb for the recession:

$$h(t) = c + a_1 / (1 + e^{(-k_1(t-t_{01}))}) + a_2 / (1 + e^{(-k_2(t-t_{02}))})$$

Fit with `scipy.optimize.curve_fit(..., method="lm")`, p0 read off the raw plot:

param	value	SE	t	p
c	14.349 m	0.0097	1483	<0.001
a_1	8.916 m	0.753	11.8	<0.001
k_1	0.512 /h	0.021	24.0	<0.001
t_{01}	31.52 h	0.091	348	<0.001
a_2	-3.721 m	0.759	-4.9	<0.001
k_2	0.283 /h	0.014	20.6	<0.001
t_{02}	37.55 h	1.058	35.5	<0.001

$R^2 = 0.9991$, $s = 0.080$ m ($n=288$, $p=7$, $dof=281$). All six shape parameters are significant. That said — being honest about the residuals — they aren't white noise: there are long same-sign runs (positive through the pre-event creep and rise, negative right after the crest). That means the two-logistic shape is a good approximation, not an exact physical model; a slow initial rise before the main pulse isn't in the functional form. Worth stating plainly if this goes into the report, rather than hiding behind the R^2 .

2. Smoothing the derivative from the fit

lab02 AI Prompts:

Differentiating the fitted function analytically (logistic derivative $k \cdot S \cdot (1-S)$) instead of taking finite differences on the raw, centimeter-rounded log:

- Raw finite-difference dh/dt peaks at 0.96 m/h at $t=29.25$ h — but it's jumpy because it's amplifying the logger's own rounding.
- The smooth derivative from the fit peaks at 1.01 m/h at $t \approx 31.3$ h, tracking the true inflection of the rise rather than a rounding artifact.
- The second-derivative panel makes the point starkest: the raw second difference is almost pure noise (± 0.15 – 0.8 m/h², driven entirely by ± 1 cm rounding), while the fitted second derivative reveals a real, clean two-lobed signal — acceleration into the rise, then deceleration through the crest and recession.

That comparison is exactly the argument the activity wants you to make in Tab 1/Tab 2: a logger hands you differences, not a derivative, and the fitted continuous $h(t)$ is what lets you report dh/dt meaningfully.

If you want, I can carry this through the rest of the lab — the quad integration (with trapezoid cross-check), the residual write-up, and the actual lab02_surname.py + dashboard HTML per Rule 0.

Prompt 2: now with the derivatives smoothed, continue with:

1. DATA PREPARATION & FINITE DIFFERENCES
2. CURVE FITTING & STATISTICAL METRICS
3. NUMERICAL INTEGRATION
4. DASHBOARD GENERATION

Claude: Both files are ready. Here's what each section does:

1. Data prep & finite differences — loads the "Sensor Log" sheet, converts timestamps to hours-elapsed ($dt=0.25$ h), computes forward/backward/central first differences and the second difference. Raw max $dh/dt = 0.960$ m/h at $t=29.25$ h — noisy, since it's amplifying the logger's 1 cm rounding.

2. Curve fitting & stats — sum-of-two-logistics model (justified in the dashboard: a single logistic can't reproduce a rise-then-recede-to-a-new-plateau shape).

`curve_fit(method="lm")` with a `p0` read off the plot. $R^2=0.9991$, $s=0.080$ m, all 7 params significant ($p<0.001$). I also wrote an automatic residual reader — it's honest that the fit is good but not perfect: residuals drift in long same-sign runs during the rise and recession (a shape issue, not sensor noise), largest residual $\approx 22\times$ the logger's resolution.

3. Integration — quad on the fitted curve gives $A = 1260.97$ m·h (abs. error $\sim 5 \times 10^{-7}$); `np.trapezoid` on the raw points gives 1260.996 m·h, a gap of -0.025 m·h ($\sim 0.002\%$), explained as quad smoothing through the fit vs. trapezoid taking the rounded points

lab02 AI Prompts:

literally. Units and what the number does/doesn't mean to the flood-control office are spelled out in the dashboard.

4. Dashboard — single self-contained HTML, no CDN/network dependency (every chart is a Python-rendered PNG embedded as base64), raw log always visible above three tabs (derivatives / fit+stats / area), tab-switching JS does nothing but toggle visibility — no computation in the browser, per Rule 0.

One thing worth flagging before you submit: the sheet has 288 readings over 3 days, not the 96-reading single day the activity text describes — the script reports n everywhere dynamically rather than hard-coding 96, but you may want to check with your instructor whether that's the intended dataset. You'll also still need to rename the files to `lab02_<yoursurname>.py/.html`, write the one-page results PDF, and export your actual prompt history — those are on you per the "no submitting what you can't explain" rule.

SCREENSHOTS

Data01.xlsx

XLSX

PDF

Analyze the past data that contains the reservoir stage record with a logger every 15 minutes. 1. get the function of the data series
2. smoothen the curve from the derivatives from the fit

Ran 9 commands, viewed 3 files, read a file >

What the log actually shows
Your `Data01.xlsx` "Sensor Log" sheet is 288 readings at 15-min spacing — three days (0 to 71.75 h from the first timestamp), not the single 96-reading day the activity sheet describes. The shape is a slow pre-event creep (14.18 → -14.5 m over 25 h), a sharp rise to a crest of -21.1 m around $t = 36$ h, then a recession that overshoots down and settles to a new plateau near 19.5 m — a classic flood pulse, not a simple monotonic fill.

1. The function of the data series
A single logistic can't produce that overshoot-and-settle behavior, so I used the lab's "sum of two logistics" candidate — one rising limb for the flood rise, a second (negative-amplitude, slower) limb for the recession:
$$h(t) = c + a_1 / (1 + e^{-k_1(t-t01)}) + a_2 / (1 + e^{-k_2(t-t02)})$$

Fitted with `scipy.optimize.curve_fit(..., method="lm")`, p0 read off the raw plot:

param	value	SE	t	p
c	14.349 m	0.0097	1483	<0.001
a_1	8.916 m	0.753	11.8	<0.001
k_1	0.512 /h	0.1	24.0	<0.001

You are out of free messages until 4:00 AM Upgrade

t01	31.52 h	0.091	348	<0.001
a_2	-3.721 m	0.759	-4.9	<0.001
k_2	0.283 /h	0.014	20.6	<0.001
t02	37.55 h	1.058	35.5	<0.001

$R^2 = 0.9991$, $s = 0.080$ m ($n=288$, $p=7$, $dof=281$). All six shape parameters are significant. That said — being honest about the residuals — they aren't white noise: there are long same-sign runs (positive through the pre-event creep and rise, negative right after the crest). That means the two-logistic shape is a good approximation, not an exact physical model; a slow initial rise before the main pulse isn't in the functional form. Worth stating plainly if this goes into the report, rather than hiding behind the R^2 .

2. Smoothing the derivative from the fit
Differentiating the fitted function analytically (logistic derivative $kS(1-S)$) instead of taking finite differences on the raw, centimeter-rounded log:

- Raw finite-difference dh/dt peaks at **0.96 m/h** at $t=29.25$ h — but it's jumpy because it's amplifying the logger's own rounding.
- The smooth derivative from the fit peaks at **1.01 m/h** at $t=31.3$ h, tracking the true inflection of the rise rather than a rounding artifact.
- The second-derivative panel makes the point starkest: the raw second difference is almost pure noise (± 0.15 – 0.8 m/h², driven entirely by ± 1 cm rounding), while the fitted second derivative reveals a real, clean two-lobed signal — acceleration into the rise, then deceleration through the crest and recession.

That comparison is exactly the argument the activity wants you to make in Tab 1/Tab 2: a logger hands you differences, not a derivative, and the fitted continuous $h(t)$ is what lets you report dh/dt meaningfully.

If you want, I can carry this through the rest of the lab — the quad integration (with trapezoid cross-check), the residual write-up, and the actual `lab02_surname.py` + dashboard HTML per Rule 0.

Smoothed derivatives

Image PNG

Download

5 hours ago

lab02 AI Prompts:

now with the derivatives smoothed, continue with:

1. DATA PREPARATION & FINITE DIFFERENCES
2. CURVE FITTING & STATISTICAL METRICS
3. NUMERICAL INTEGRATION
4. DASHBOARD GENERATION

Ran 2 commands, created a file >

Ran clean end to end. Let me sanity-check the rendered dashboard before handing it over.

Ran a command `cd /mnt/user-data/outputs && python3 -c "import rehtml = o..."` >

All 5 figures embedded, no unfilled template fields, tab-switch JS present. Let me render it to a screenshot-like check by viewing one panel image directly to confirm quality.

Ran 2 commands, viewed a file, read a file >

Both files are ready. Here's what each section does:

1. Data prep & finite differences — loads the "Sensor Log" sheet, converts timestamps to hours-elapsed ($dt=0.25$ h), computes forward/backward/central first differences and the second difference. Raw max $dh/dt = 0.960$ m/h at $t=29.25$ h — noisy, since it's amplifying the logger's 1 cm rounding.

2. Curve fitting & stats — sum-of-two-logistics model (justified in the dashboard: a single logistic can't reproduce a rise-then-recede-to-a-new-plateau shape). `curve_fit(method="lm")` with a p0 read off the plot. $R^2=0.9991$, $s=0.080$ m, all 7 params significant ($p<0.001$). I also wrote an automatic residual reader — it's honest that the fit is good but not perfect: residuals drift in long same-sign runs during the rise and recession (a shape issue, not sensor noise), largest residual $\approx 2\times$ the logger's resolution.

3. Integration — `quad` on the fitted curve gives $A = 1260.97$ m-h (abs. error $\sim 5\times 10^{-7}$); `np.trapezoid` on the raw points gives 1260.996 m-h, a gap of -0.025 m-h ($\sim 0.002\%$), explained as quad smoothing through the fit vs. trapezoid taking the rounded points literally. Units and what the number does/doesn't mean to the flood-control office are spelled out in the dashboard.

4. Dashboard — single self-contained HTML, no CDN/network dependency (every chart is a Python-rendered PNG embedded as base64), raw log always visible above three tabs (derivatives / fit+stats / area), tab-switching JS does nothing but toggle visibility — no computation in the browser, per Rule 0.

One thing worth flagging before you submit: the sheet has **288 readings over 3 days**, not the 96-reading single day the activity text describes — the script reports `n` everywhere dynamically rather than hard-coding 96, but you may want to check with your instructor whether that's the intended dataset. You'll also still need to rename the files to `lab02_<yoursurname>.py / .html`, write the one-page results PDF, and export your actual prompt history — those are on you per the "no submitting what you can't explain" rule.

Lab02 analysis
PY

Download

Lab02 dashboard
Code HTML

Download

Download all

You are out of free messages until 4:00 AM

Upgrade

Cube pt.2:

Prompt: with the given python code, add:

Improve the existing Python solver by adding the following features, similar to RISA or STAAD:

1. Add support conditions at the bottom nodes. For now, use pinned supports.
2. Add a beta angle.
- 3 Add local axes and global
4. Add degrees of freedom (DOF) for each node.
5. Add degrees of freedom for beam pinned at x direction moment at z direction
6. Show Pinned symbol if beam is pinned.
7. Update the structural diagram.
8. Update the Excel output file.

Save the updated Python solver as Rev. 1

lab02 AI Prompts:

Claude: Rev. 1 is ready — I ran it end-to-end to confirm it executes cleanly. Here's what was added:

1. **Supports** – bottom nodes (Y=0: N1–N4) get pinned supports (translations fixed, rotations free); everything else is auto-tagged "Free."
2. **Beta angle** – a per-member `beta_angle` dict (default 0°); the four vertical columns are set to 45° as a working example.
3. **Local + global axes** – `compute_local_axes()` builds local x/y/z per member using the standard STAAD/RISA convention (reference vector = global Y, or global Z if the member is vertical), then rotates y/z by beta. Local triads (red/green/purple) are drawn at every member midpoint; a black global X/Y/Z triad is drawn off to the side.
4. **DOF numbering** – 6 DOF/node (UX, UY, UZ, RX, RY, RZ), numbered 1–48 across the structure, in a Node DOF Table.
5. **Pinned beam releases** – a `member_releases` dict; the top-face beams (M5–M8) are flagged pinned, releasing RX (torsion) at the start node and RZ (bending) at the end node — easy to reassign to other members.
6. **Pin symbols** – open magenta circles drawn on the diagram at any released end.
7. **Diagram** – rebuilt to overlay supports (orange triangles), local axis triads, the global axis triad, and pin symbols on top of the original node/member plot, with a proper legend.
8. **Excel workbook** (`cube_structure_6m_Rev1.xlsx`) now has 7 tabs: Nodes, Member Incidences, Node Summary, Node DOF Table, Support Conditions, Local Axes & Beta Angle, and Member End Releases.

All the "which members are pinned," "which get a beta angle," and "which nodes are supported" choices are driven by small config blocks near the top of the script (`SUPPORT_NODES`, `beta_angle`, `PINNED_MEMBERS`), so you can adjust them without touching the geometry or plotting logic.

SCREENSHOTS

CUBE.py
163 lines

PY

with the given python code, add:
Improve the existing Python solver by adding the following features, similar to RISA or STAAD:

1. Add support conditions at the bottom nodes. For now, use pinned supports.
2. Add a beta angle. 3 Add local axes and global
3. Add degrees of freedom (DOF) for each node.
4. Add degrees of freedom for beam pinned at x direction moment at z direction
5. Show Pinned symbol if beam is pinned.
6. Update the structural diagram.
7. Update the Excel output file.

Save the updated Python solver as Rev. 1

Ran 2 commands, created a file

Ran cleanly. Let me verify the diagram Image and Excel workbook look right before delivering.

Viewed 2 files, edited 2 files, ran 2 commands · 1 note

Rev. 1 is ready — I ran it end-to-end to confirm it executes cleanly. Here's what was added:

1. **Supports** - bottom nodes (Y=0: N1-N4) get pinned supports (translations fixed, rotations free); everything else is auto-tagged "Free."

2. **Beta angle** - a per-member `beta_angle` dict (default 0°); the four vertical columns are set to 45° as a working example.

3. **Local + global axes** - `compute_local_axe` builds local x/y/z per member using the

3. **Local + global axes** - `compute_local_axes()` builds local x/y/z per member using the standard STAAD/RISA convention (reference vector = global Y, or global Z if the member is vertical), then rotates y/z by beta. Local triads (red/green/purple) are drawn at every member midpoint; a black global X/Y/Z triad is drawn off to the side.

4. **DOF numbering** - 6 DOF/node (UX, UY, UZ, RX, RY, RZ), numbered 1-48 across the structure, in a `Node DOF Table`.

5. **Pinned beam releases** - a `member_releases` dict; the top-face beams (M5-M8) are flagged pinned, releasing RX (torsion) at the start node and RZ (bending) at the end node — easy to reassign to other members.

6. **Pin symbols** - open magenta circles drawn on the diagram at any released end.

7. **Diagram** - rebuilt to overlay supports (orange triangles), local axis triads, the global axis triad, and pin symbols on top of the original node/member plot, with a proper legend.

8. **Excel workbook** (`cube_structure_6m_Rev1.xlsx`) now has 7 tabs: Nodes, Member Incidences, Node Summary, Node DOF Table, Support Conditions, Local Axes & Beta Angle, and Member End Releases.

All the "which members are pinned," "which get a beta angle," and "which nodes are supported" choices are driven by small config blocks near the top of the script (`SUPPORT_NODES` , `beta_angle` , `PINNED_MEMBERS`), so you can adjust them without touching the geometry or plotting logic.

Cube rev1
PY

Download

Cube structure 6m rev1
Spreadsheet: XLSX

Download

Cube structure 3d rev1
Image: PNG

Download

Download all